

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Учебно - ознакомительная практика
по дисциплине: Основы программирования

Выполнил: ст. группы ПВ-221
Дорошенко Владимир Юрьевич
Проверили:
Черников Сергей Викторович
Новожен Никита Викторович

Белгород 2023 г.

Содержание

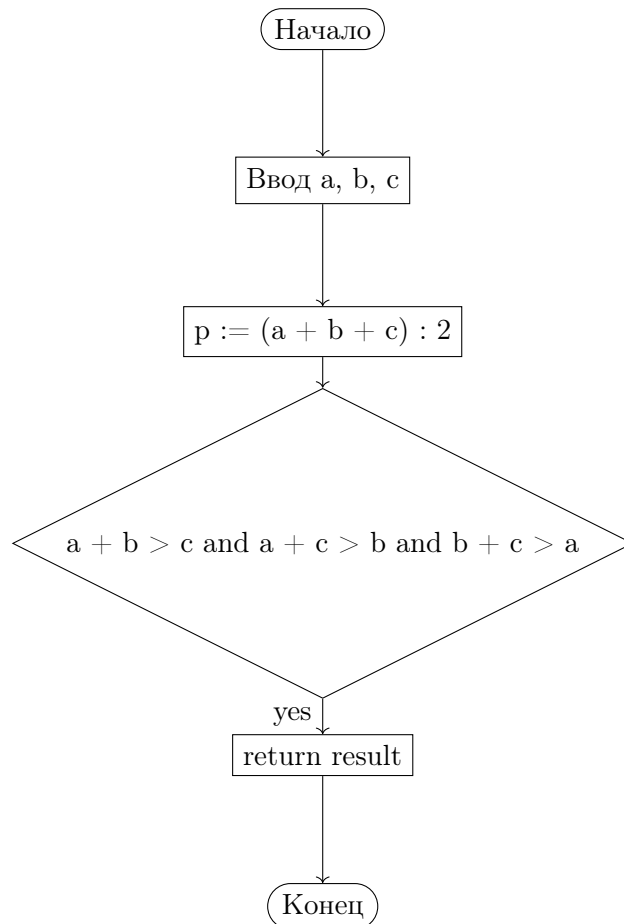
1	Учебно - ознакомительная практика	3
1.1	Тема 1. Линейные алгоритмы	3
1.2	Тема 2. Разветвляющиеся алгоритмы	5
1.3	Тема 3. Циклические и итерационные алгоритмы	9
1.4	Тема 4. Простейшие операции над массивами	11
1.5	Тема 5. Векторы и матрицы	13
1.6	Тема 6. Линейный поиск	15
1.7	Тема 7. Арифметика	19
1.8	Тема 8. Геометрия и теория множеств	23
1.9	Тема 9. Линейная алгебра и сжатие информации	28
1.10	Тема 10. Алгоритм обработки символьной информации	33
1.11	Тема 11. Аналитическая геометрия	37
1.12	Тема 12. Кривые второго порядка на плоскости	39
1.13	Тема 13. Графическое решение систем уравнений	40
1.14	Тема 14. Плоскость в трехмерном пространстве	42
1.15	Тема 15. Поверхность второго порядка в трехмерном пространстве	43
1.16	Заключение	44

1 Учебно - ознакомительная практика

1.1 Тема 1. Линейные алгоритмы

Треугольник ABC задан длинами своих сторон. Найти длину высоты, опущенной из вершины A .

Блок - схема:



Обоснование: Данная программа реализует функцию `triangleHeightLength`, которая вычисляет длину высоты, опущенной из вершины A в треугольнике ABC . Функция принимает три параметра a , b и c , которые представляют длины сторон треугольника.

Код программы:

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <assert.h>
4
5 // возвращает длину высоты в треугольнике ABC, опущенной из вершины A.
6 double triangleHeightLength(double a, double b, double c) {
7     double p = (a + b + c) / 2;
8     if (a + b > c && a + c > b && b + c > a) {
9         return 2 * sqrt(p * (p - a) * (p - b) * (p - c)) / a;
10    }
11 }
```

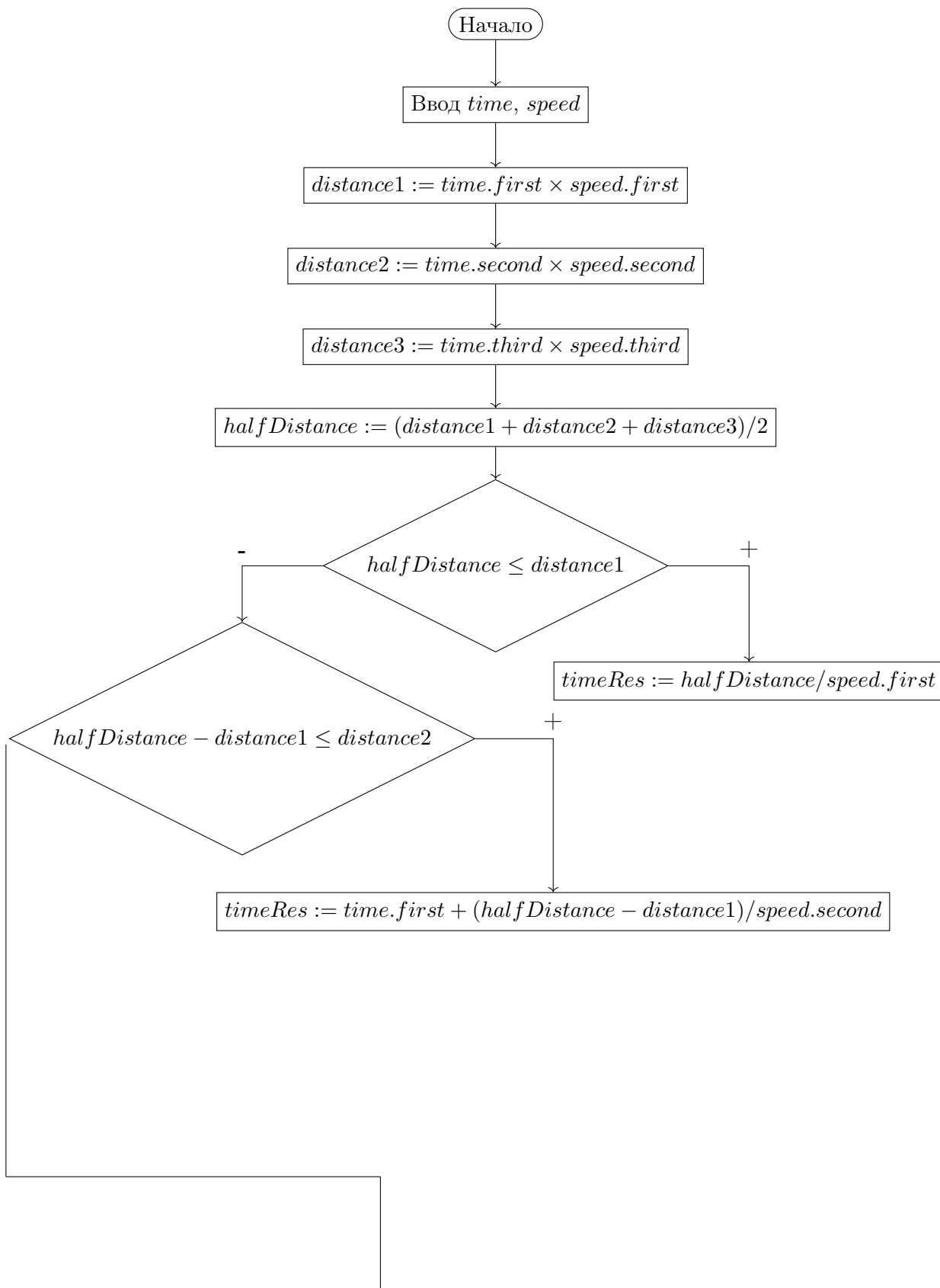
Тесты:

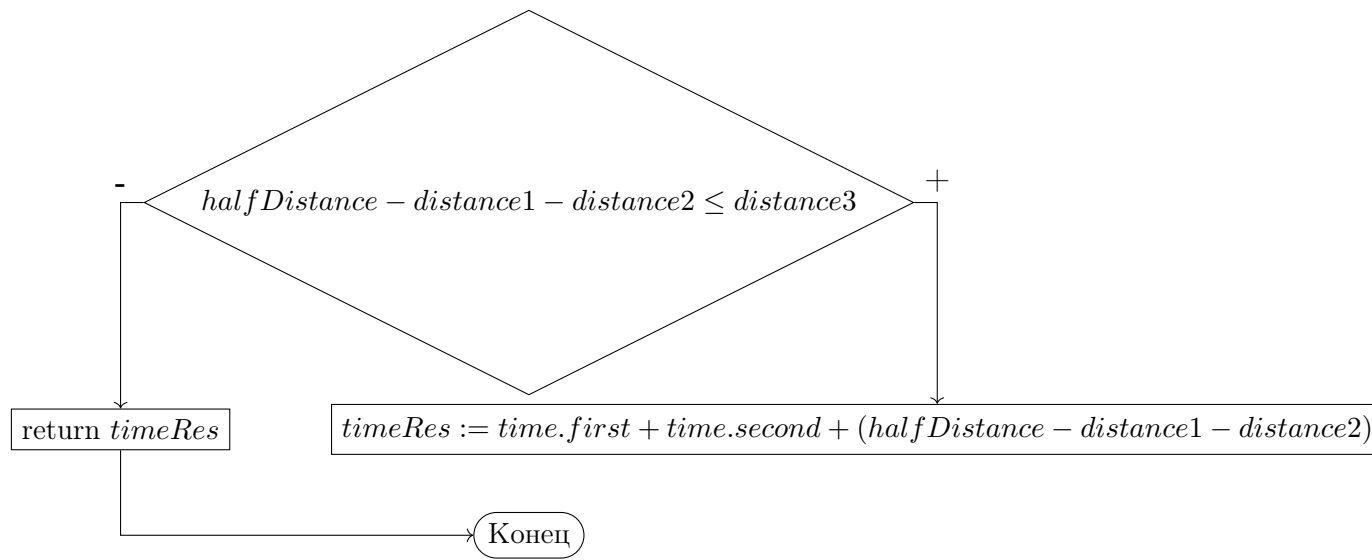
```
1
2 void test1_17_1() {
3     int rez = triangleHeightLength(4, 2.5, 3);
4     assert(rez == 1);
5 }
6
7 void test1_17_2() {
8     float rez = triangleHeightLength(3, 7, 9);
9     assert(rez == 0);
10 }
11
12 void test1_17_3() {
13     float rez = triangleHeightLength(8, 7.2, 9);
14     assert(round(rez * 10) / 10 == 6.9);
15 }
16
17 void test() {
18     test1_17_1();
19     test1_17_2();
20     test1_17_3();
21 }
22
23 int main() {
24     test();
25
26     return 42;
27 }
```

1.2 Тема 2. Разветвляющиеся алгоритмы

Путник двигался t_1 часов со скоростью v_1 , затем t_2 часов — со скоростью v_2 и t_3 часов — со скоростью v_3 . За какое время он одолел первую половину пути, после чего запланировал привал?

Блок - схема:





Обоснование: Данная программа реализует функцию `getTimeHalfDistance`, которая вычисляет оптимальное время прохождения первой половины пути для трех людей. Функция принимает два параметра типа `People` - `time` и `speed`. Структура `People` содержит три поля типа `float`: `first`, `second` и `third`, которые представляют время, затраченное каждым человеком на прохождение соответствующей части пути, и скорость каждого человека.

Код программы:

```
1 //структура путника(человека), для разных частей пути
2 struct People {
3     float first;
4     float second;
5     float third;
6 };
7
8 // возвращает оптимальное время прохождения первой половины пути
9 float getTimeHalfDistance(People time, People speed) {
10     float timeRes = 0;
11     float distance1 = time.first * speed.first;
12     float distance2 = time.second * speed.second;
13     float distance3 = time.third * speed.third;
14     float halfDistance = (distance1 + distance2 + distance3) / 2;
15     if (halfDistance <= distance1) {
16         timeRes = halfDistance / speed.first;
17     } else if (halfDistance - distance1 <= distance2) {
18         timeRes = time.first + (halfDistance - distance1) / speed.second;
19     } else if (halfDistance - distance1 - distance2 <= distance3) {
20         timeRes = time.first + time.second + (halfDistance - distance1 -
21         distance2) / speed.third;
22     }
23     return timeRes;
24 }
```

Тесты:

```
1 void test2_17_1() {
2     People time = {9, 8, 6};
3     People speed = {10, 15, 5};
4     int rez = getTimeHalfDistance(time, speed);
5     assert(rez == 11);
6 }
7
8 void test2_17_2() {
9     People time = {19, 14, 1};
10    People speed = {8, 2, 7};
11    float rez = getTimeHalfDistance(time, speed);
12    assert(round(rez * 10) / 10 == 11.7);
13 }
14 void test2_17_3() {
15     People time = {15, 5, 19};
16     People speed = {8, 17, 7};
17     float rez = getTimeHalfDistance(time, speed);
18     assert(round(rez * 10) / 10 == 17.9);
19 }
20 int main() {
21     test2_17_1();
22     test2_17_2();
23     test2_17_3();
24
25     return 42;
26 }
```

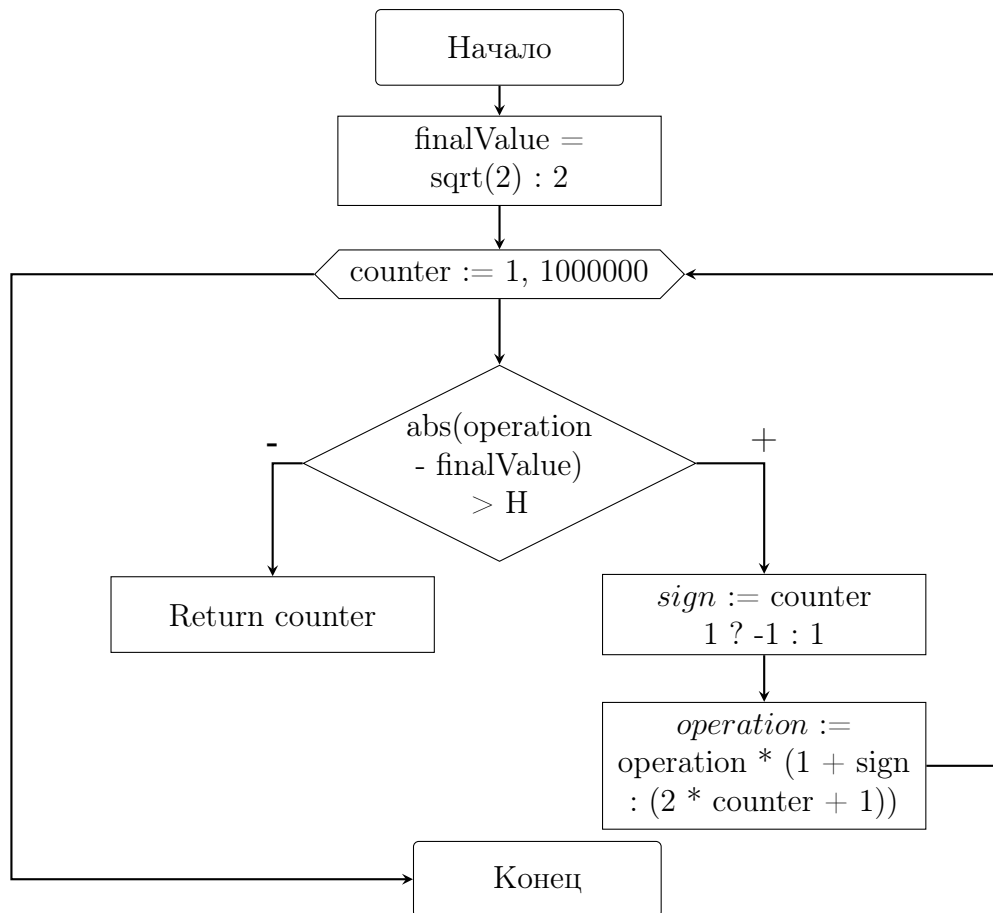
1.3 Тема 3. Циклические и итерационные алгоритмы

Сколько сомножителей надо взять в произведении:

$$\prod_{k=1}^{\infty} \left(1 + \frac{(-1)^k}{2k+1} \right) = \frac{\sqrt{2}}{2}$$

чтобы равенство выполнялось до шестой значащей цифры, то есть с погрешностью не более 10^6 ?

Блок - схема:



Обоснование:

Данная программа реализует функцию countFactors, которая вычисляет количество сомножителей в заданном выражении. Функция не принимает параметров.

Код программы:

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <assert.h>
4
5 #define H 0.000001
6
7 // возвращает количество сомножителей в данном выражении
8 int countFactors() {
9     long double finalValue = sqrt(2) / 2, operation = 1;
10    for (int counter = 1; counter < 1000000; ++counter) {
11        if (abs(operation - finalValue) > H) {
12            int sign = counter & 1 ? -1 : 1;
13            operation = operation * (long double) (1 + (long double) sign / (2
14            * counter + 1));
15        } else {
16            return counter;
17        }
18    }
19 }
20
21 int main() {
22     printf(countFactors());
23
24     return 42;
25 }
```

Тесты:

```
1 void test3_17_1() {
2     assert(countFactors() == 88390);
3 }
4 void test() {
5     test3_17_1();
6 }
7
8 int main() {
9     test();
10
11     return 42;
12 }
```

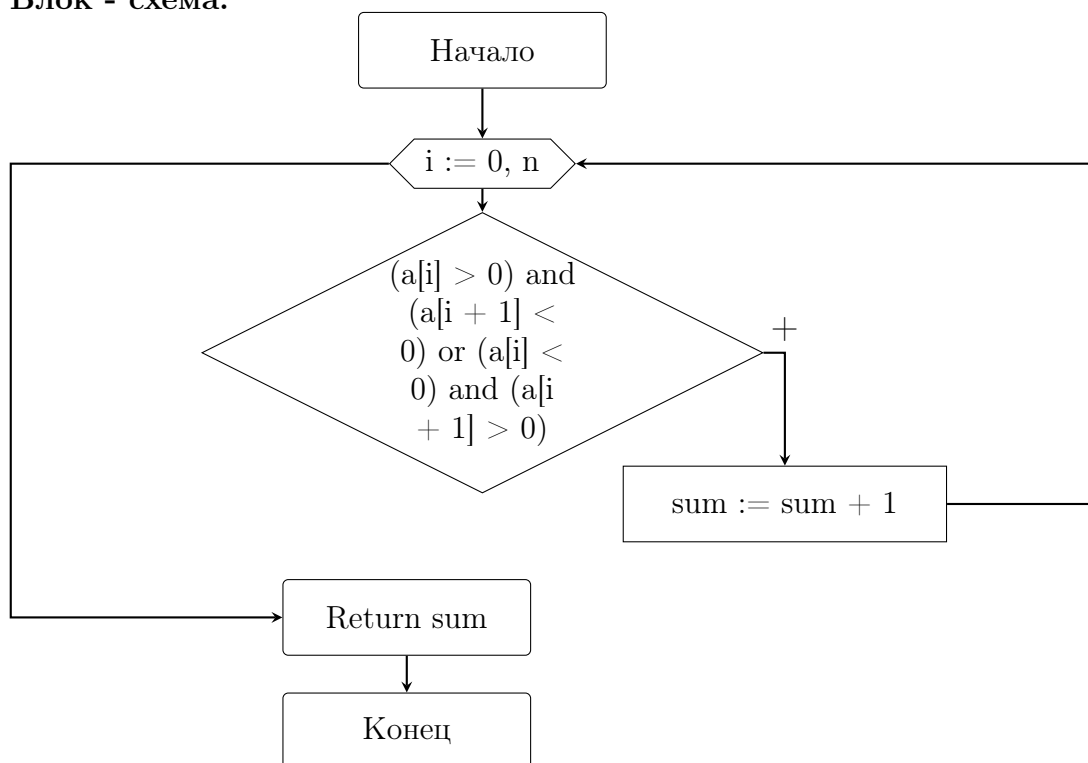
Результаты работы программы:

```
88390
Process finished with exit code 42
```

1.4 Тема 4. Простейшие операции над массивами

В массиве $Z(m)$ найти число чередований знака, то есть число переходов с минуса на плюс или с плюса на минус (как известно, нуль не имеет знака).

Блок - схема:



Обоснование Данная программа реализует функцию `sumPlusOrMinus`, которая принимает массив `a` целых чисел длиной `n` и возвращает сумму переходов с положительных чисел на отрицательные или с отрицательных на положительные числа в этом массиве.

Код программы:

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <assert.h>
4
5 // возвращает в массиве a сумму переходов с минуса на плюс или с плюса на минус
6 int sumPlusOrMinus(int *a, int n) {
7     int sum = 0;
8     for (int i = 0; i < n; i++) {
9         if ((a[i] > 0) && (a[i + 1] < 0) || (a[i] < 0) && (a[i + 1] > 0)) {
10             sum++;
11         }
12     }
13
14     return sum;
15 }
```

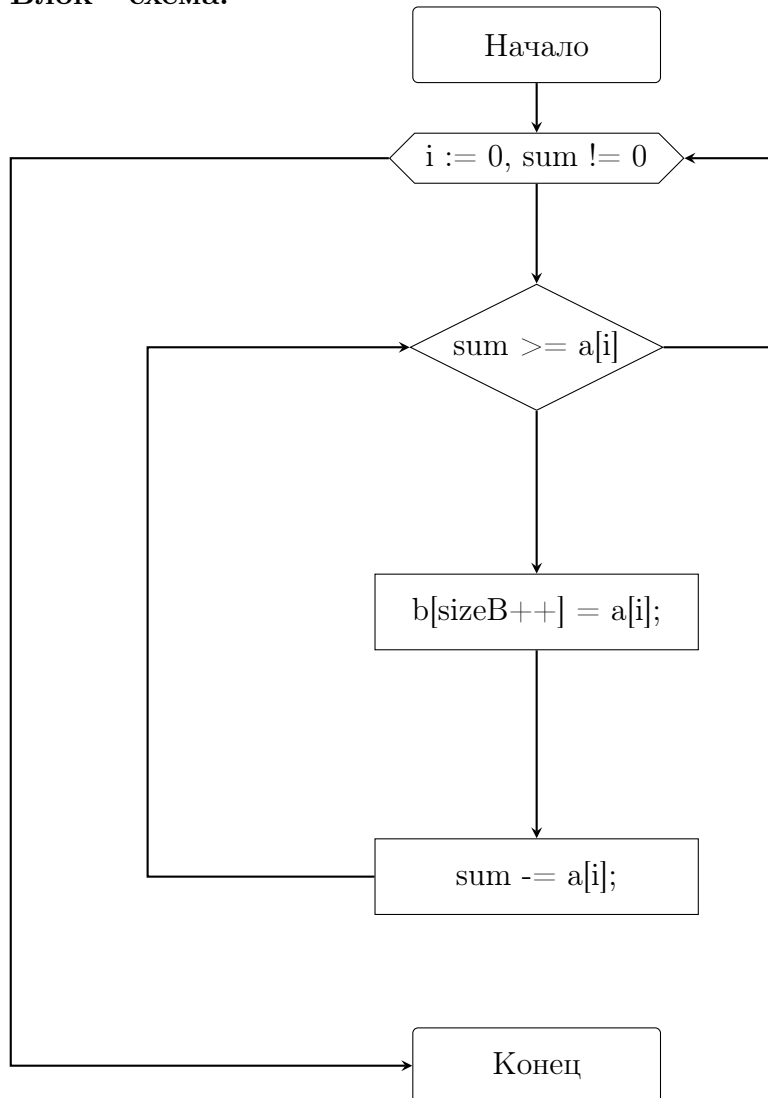
Тесты:

```
1 void test4_17_1() {
2     int a[] = {0, -2, 0, -1, 0, 2, -1, 0, 0, 3, 2, -3};
3     assert(sumPlusOrMinus(a, 12) == 3);
4 }
5
6 void test4_17_2() {
7     int a[] = {0, -2, 0, -10, 2, -1, 0, 0, 3, 2, -3, 4};
8     assert(sumPlusOrMinus(a, 12) == 4);
9 }
10
11 void test4_17_3() {
12     int a[] = {-1, 2, 0, 0, -33, 9, -4, 0, 3, 0, -1, 2, -5};
13     assert(sumPlusOrMinus(a, 13) == 5);
14 }
15
16 void test() {
17     test4_17_1();
18     test4_17_2();
19     test4_17_3();
20 }
21
22 int main() {
23     test();
24
25     return 42;
26 }
```

1.5 Тема 5. Векторы и матрицы

В массиве $K(n)$ в порядке убывания представлены достоинства денежных знаков (купюр и монет) валютной системы некоторой страны. Реализовать выдачу в этой системе заданной суммы минимальным числом денежных знаков.

Блок - схема:



Обоснование: Данная программа реализует функцию giveMoney, которая принимает массив a с доступными денежными номиналами, массив b для записи выдачи и размер массива b, а также сумму sum, которую необходимо выдать. Функция записывает в массив b минимальное количество денежных номиналов, чтобы сумма выданных денег равнялась sum.

Код программы:

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <assert.h>
4
5 // записывает в массив b выдачу заданной суммы минимальным числом денежных знаков
6 void giveMoney(int *a, int *b, int sizeB, int sum) {
7     for (size_t i = 0; sum != 0; ++i) {
8         while (sum >= a[i]) {
9             b[sizeB++] = a[i];
10            sum -= a[i];
11        }
12    }
13 }
```

Тесты:

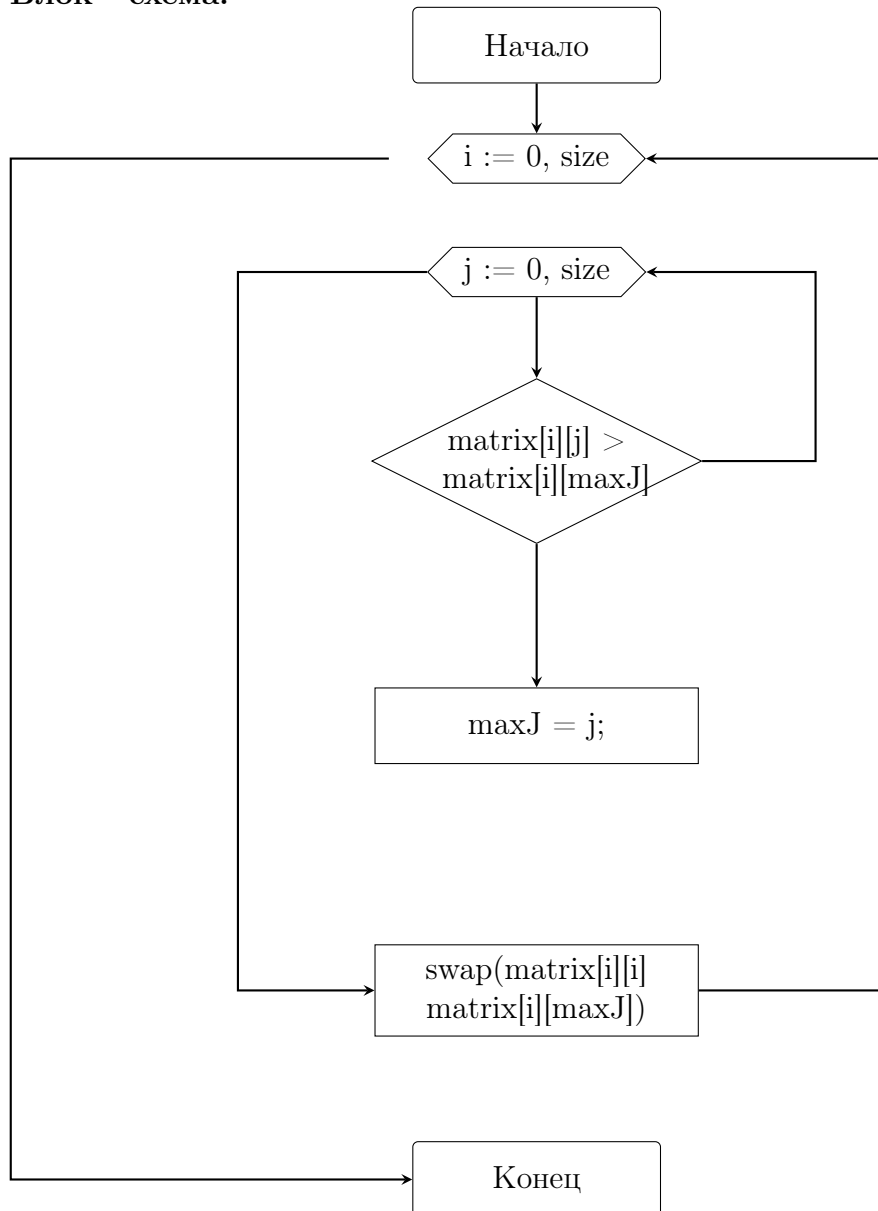
```
1 void test5_17_1() {
2     int a[] = {5000, 1000, 500, 200, 100, 50, 20, 10, 5, 2, 1};
3     int b[] = {1000, 1000};
4     int res[] = {0};
5     giveMoney(a, res, 2, 2000);
6
7     for (size_t i = 0; i < sizeof(res) / sizeof(res[0]); i++) {
8         assert(b[i] == res[i]);
9     }
10 }
11
12 void test5_17_2() {
13     int a[] = {5000, 1000, 500, 200, 100, 50, 20, 10, 5, 2, 1};
14     int b[] = {10, 5};
15     int res[] = {0};
16     giveMoney(a, res, 2, 15);
17
18     for (size_t i = 0; i < sizeof(res) / sizeof(res[0]); i++) {
19         assert(b[i] == res[i]);
20     }
21 }
22
23 void test5_17_3() {
24     int a[] = {5000, 1000, 500, 200, 100, 50, 20, 10, 5, 2, 1};
25     int b[] = {2, 2};
26     int res[] = {0};
27     giveMoney(a, res, 2, 4);
28
29     for (size_t i = 0; i < sizeof(res) / sizeof(res[0]); i++) {
30         assert(b[i] == res[i]);
31     }
32 }
33 int main() {
34     test5_17_1();
35     test5_17_2();
36     test5_17_3();
37 }
```

1.6 Тема 6. Лине́йный поиск

В каждой строке матрицы $A(n, n)$ найти наибольший элемент и поменять его местами с соответствующим диагональным элементом



Блок - схема:



Обоснование: Данная программа реализует функцию swapDiagonalMax, которая принимает двумерный массив matrix, представляющий собой матрицу, и целое число size, определяющее размер матрицы. Функция обменивает наибольший элемент в каждой строке с соответствующим элементом на главной диагонали.

Код программы:

```
1 #include <stdio.h>
2 #include <assert.h>
3
4 #define MAX_SIZE 100000000
5
6 //обменивает в строке матрицы наибольший элемент с соответствующим местом на диагональным
   элементом
7 void swapDiagonalMax(int matrix[][MAX_SIZE], int size) {
8     for (int i = 0; i < size; i++) {
9         int maxJ = 0;
10        for (int j = 0; j < size; j++) {
11            if (matrix[i][j] > matrix[i][maxJ]) {
12                maxJ = j;
13            }
14        }
15
16        int temp = matrix[i][i];
17        matrix[i][i] = matrix[i][maxJ];
18        matrix[i][maxJ] = temp;
19    }
20 }
```

Тесты:

```
1 void test6_17_1() {
2     int matrix[][MAX_SIZE] = {{1, 2, 3},
3                                {4, 5, 6},
4                                {7, 8, 9}};
5     int res[][MAX_SIZE] = {{3, 2, 1},
6                             {4, 6, 5},
7                             {7, 8, 9}};
8
9     swapDiagonalMax(matrix, sizeof(matrix) / sizeof(matrix[0][0]));
10    for (int i = 0; i < sizeof(matrix) / sizeof(matrix[0][0]); i++) {
11        for (int j = 0; j < sizeof(matrix) / sizeof(matrix[i]); ++j) {
12            assert(matrix[i][j] == res[i][j]);
13        }
14    }
15 }
16
17 void test6_17_2() {
18     int matrix[][MAX_SIZE] = {{3, 2, 3, 5},
19                                {4, 5, 10},
20                                {7, 8, 9, 10, 11}};
21     int res[][MAX_SIZE] = {{5, 2, 3, 3},
22                             {4, 10, 5},
23                             {7, 8, 11, 10, 9}};
24
25     swapDiagonalMax(matrix, sizeof(matrix) / sizeof(matrix[0][0]));
26    for (int i = 0; i < sizeof(matrix) / sizeof(matrix[0][0]); i++) {
27        for (int j = 0; j < sizeof(matrix) / sizeof(matrix[i]); ++j) {
28            assert(matrix[i][j] == res[i][j]);
29        }
30    }
31 }
```

```

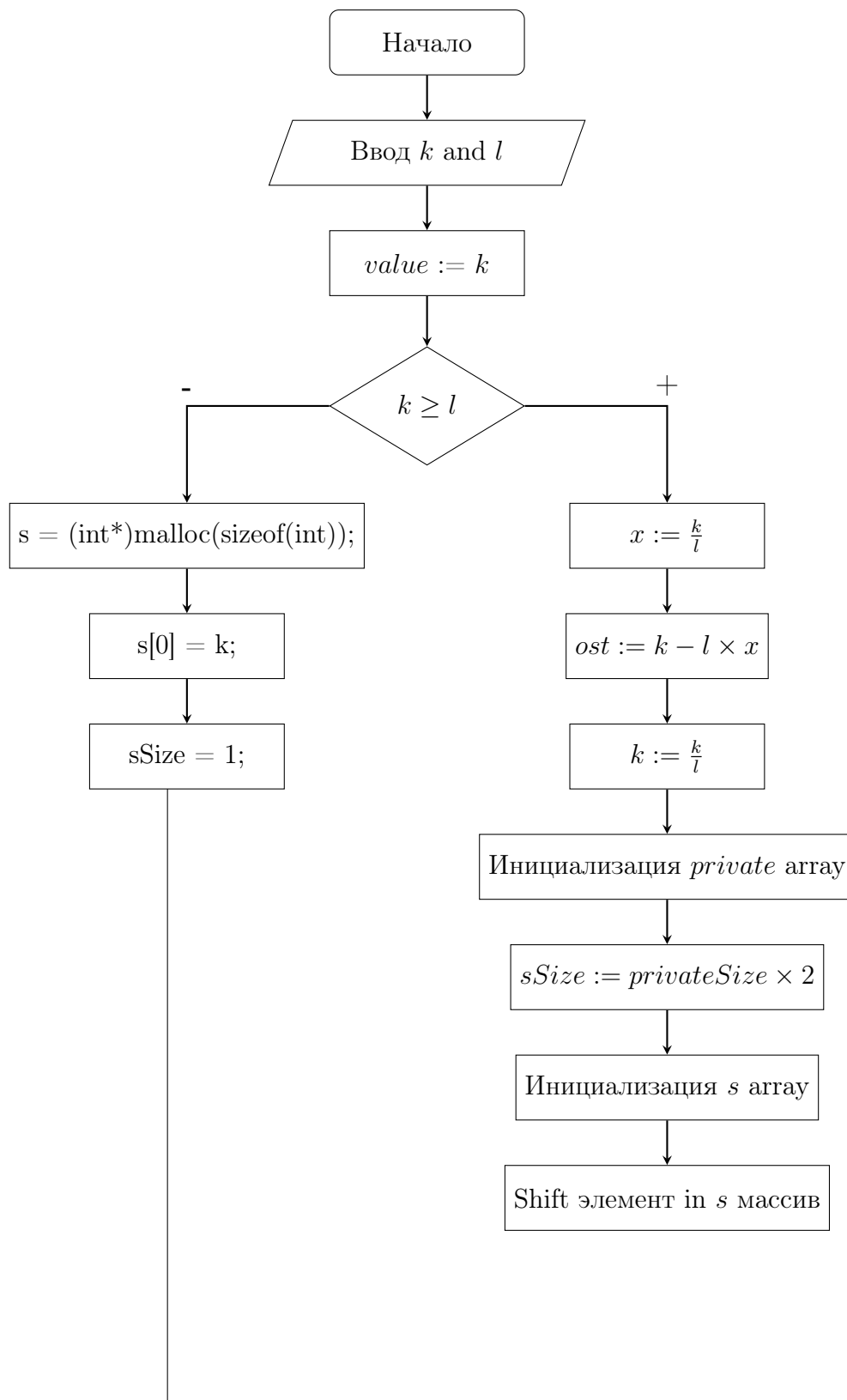
1
2 void test6_17_3() {
3     int matrix[][MAX_SIZE] = {{3, 7, 2, 9},
4                                 {4, 3, 6, 8},
5                                 {7, 8, 5, 5},
6                                 {5, 8, 90, 0}};
7     int res[][MAX_SIZE] = {{9, 7, 2, 3},
8                             {4, 8, 6, 3},
9                             {7, 5, 8, 5},
10                            {5, 8, 0, 90}};
11     swapDiagonalMax(matrix, sizeof(matrix) / sizeof(matrix[0][0]));
12     for (int i = 0; i < sizeof(matrix) / sizeof(matrix[0][0]); i++) {
13         for (int j = 0; j < sizeof(matrix) / sizeof(matrix[i]); ++j) {
14             assert(matrix[i][j] == res[i][j]);
15         }
16     }
17 }
18
19 void test() {
20     test6_17_1();
21     test6_17_2();
22     test6_17_3();
23 }
24
25 int main() {
26     test();
27
28     return 42;
29 }

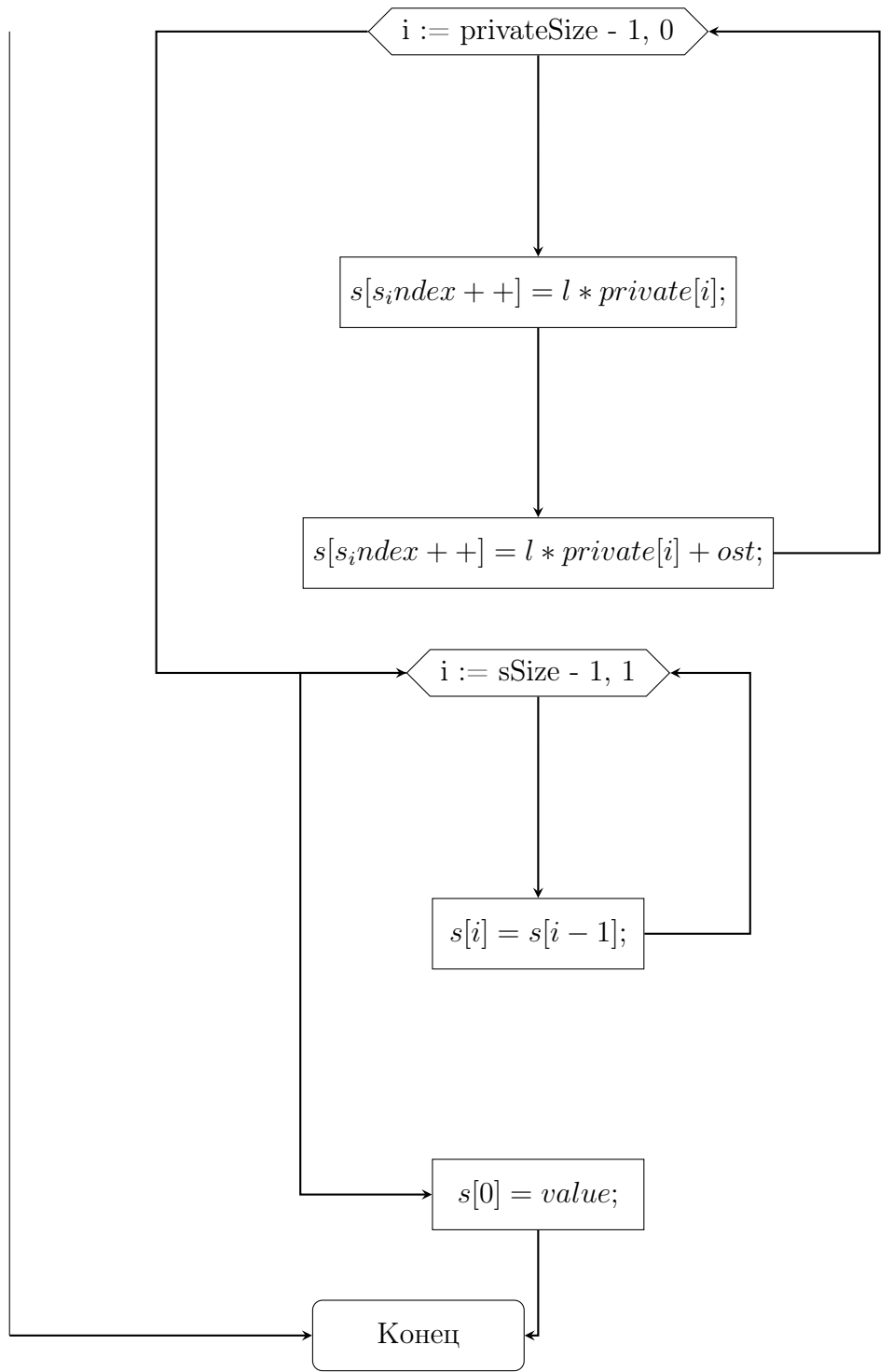
```

1.7 Тема 7. Арифметика

Вывести столбиком пример на деление с остатком двух заданных натуральных чисел k и l .

Блок - схема:





Обоснование: Данная программа реализует функцию printProbel и print, а также содержит функцию main, которая взаимодействует с пользователем, считывает значения k и l с помощью функции scanf и выполняет ряд вычислений и печати результатов.

Код программы:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // печатает количество пробелов относительно заданного k
5 void printProbel(int value, char s) {
6     while (value != 0) {
7         value /= 10;
8         printf("%c", s);
9     }
10 }
11
12 // печатает количество нижних подчеркиваний относительно заданного k
13 void print(int value, char s) {
14     while (value != 0) {
15         printf("%c", s);
16         value /= 10;
17     }
18 }
19
20 int main() {
21     int k, l;
22     scanf("%d %d", &k, &l);
23
24     int ost, x, remember_k;
25
26     int* private;
27     int* s;
28     int privateSize;
29     int sSize;
30
31     int value = k;
32
33     if (k >= 1) {
34         x = k / l;
35         ost = k - l * x;
36         k /= l;
37         remember_k = k;
38         private = (int*) malloc(sizeof(int) * k);
39         privateSize = 0;
40         while (k > 0) {
41             private[privateSize++] = k % 10;
42             k /= 10;
43         }
44
45         sSize = privateSize * 2;
46         s = (int*) malloc(sizeof(int) * sSize);
47         int s_index = 0;
48         for (int i = privateSize - 1; i >= 0; i--) {
49             s[s_index++] = l * private[i];
50             s[s_index++] = l * private[i] + ost;
51         }
52
53         for (int i = sSize - 1; i > 0; i--) {
54             s[i] = s[i - 1];
55         }
56     }
```

```

56     s[0] = value;
57 } else {
58     s = (int*) malloc(sizeof(int));
59     s[0] = k;
60     sSize = 1;
61 }
62
63 printf("%d    |%d__\n", value, 1);
64 printProbel(value, ' ');
65
66 printf("    |%d\n", remember_k);
67 for (int i = 1; i < sSize; i++) {
68     if (i & 1) {
69         printf("-");
70         print(value, '_');
71         printf("%d_\n", s[i]);
72     } else {
73         print(value, '_');
74         printf("_%d_\n", s[i]);
75     }
76 }
77 printf("ostatok  =  %d\n", ost);
78
79 free(private);
80 free(s);
81
82 return 42;
83 }

```

Результаты работы программы:

```

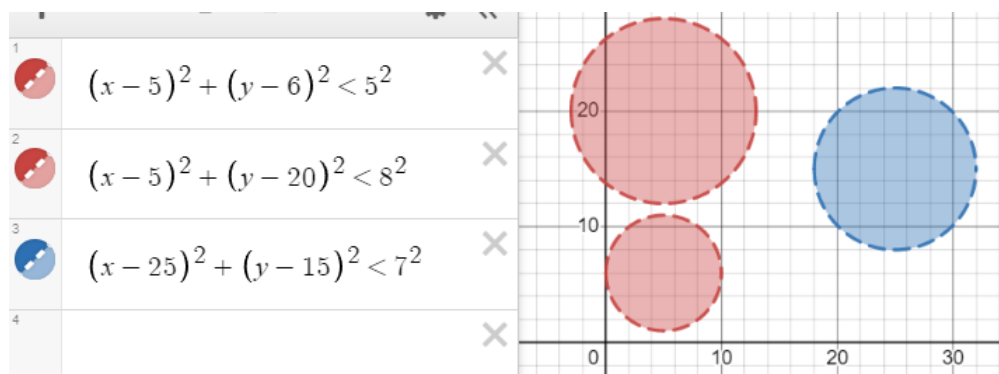
471 2
471    |2__
        |235
-___4_
----7_
-___6_
----11_
-___10_
ostatok  =  1

Process finished with exit code 42

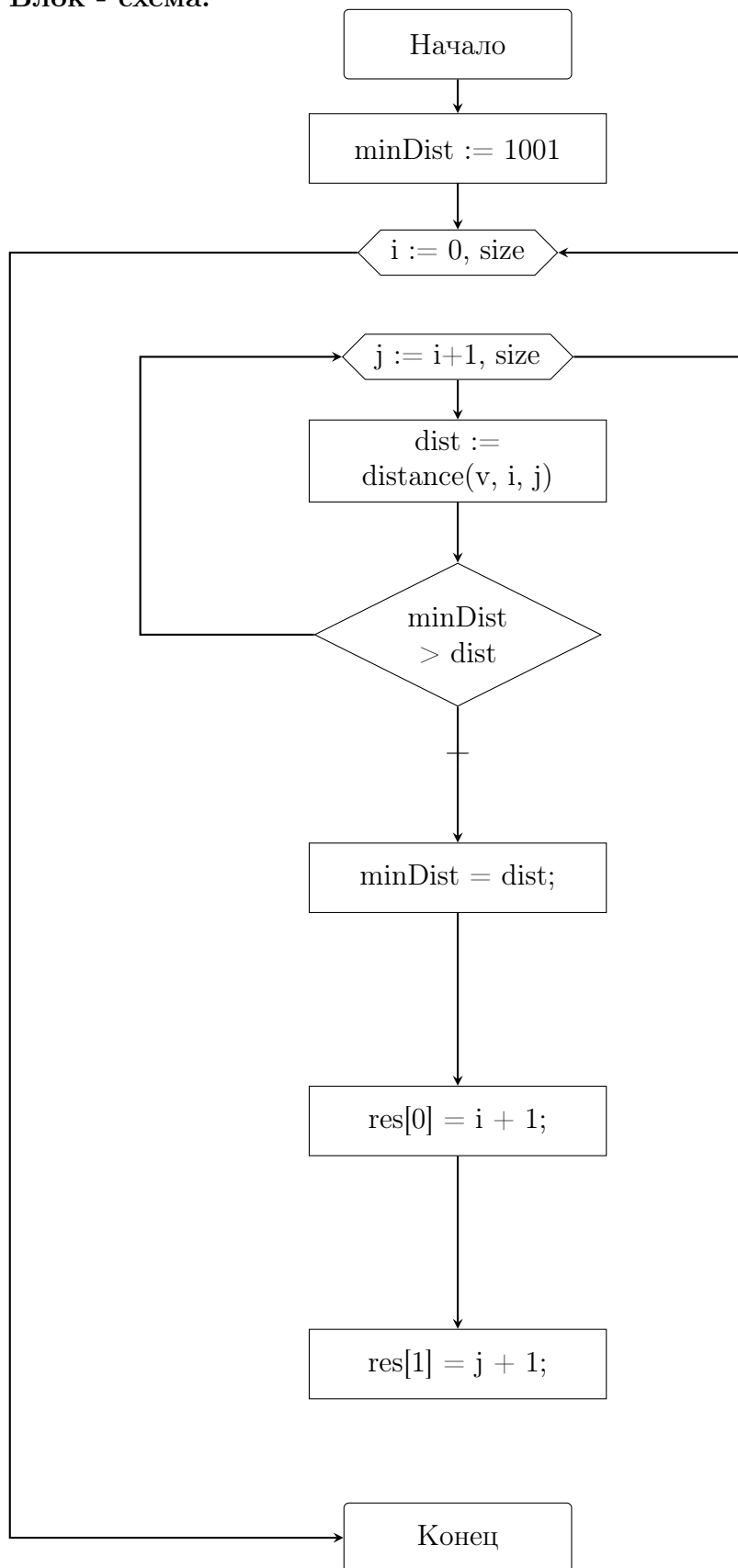
```

1.8 Тема 8. Геометрия и теория множеств

На плоскости задано n произвольно расположенных окружностей. Найти пару наименее удаленных из них.



Блок - схема:



Обоснование: Данная программа реализует функции `distance` и `findMinPairCircle`, а также содержит функцию `main`, которая взаимодействует с пользователем, считывает данные о координатах и радиусах окружностей, находит пару окружностей с минимальным расстоянием между ними и выводит их индексы.

Код программы:

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <assert.h>
4 #include <stdlib.h>
5
6 // структура окружности с координатами и радиусом
7 struct circle {
8     double x;
9     double y;
10    double R;
11 };
12
13 // возвращает расстояние между окружностями
14 double distance(struct circle* v, int i, int j) {
15     return sqrt((v[i].x - v[j].x) * (v[i].x - v[j].x) + (v[i].y - v[j].y) * (v[i].y - v[j].y)) - v[i].R - v[j].R;
16 }
17
18 // ищет минимальное расстояние между двумя окружностями и возвращает массив с индексами
19 int* findMinPairCircle(struct circle* v, int size) {
20     double minDist = 1001;
21     int* res = (int*)malloc(sizeof(int) * 2);
22     for (int i = 0; i < size; i++) {
23         for (int j = i + 1; j < size; j++) {
24             double dist = distance(v, i, j);
25             if (minDist > dist) {
26                 minDist = dist;
27                 res[0] = i + 1;
28                 res[1] = j + 1;
29             }
30         }
31     }
32     return res;
33 }
34
35 int main() {
36     int n;
37     scanf("%d", &n);
38
39     struct circle* v = (struct circle*)malloc(sizeof(struct circle) * n);
40     for (int i = 0; i < n; i++) {
41         scanf("%lf %lf %lf", &(v[i].x), &(v[i].y), &(v[i].R));
42     }
43
44     int* minPair = findMinPairCircle(v, n);
45     printf("%d %d\n", minPair[0], minPair[1]);
46
47     free(v);
48     free(minPair);
49
50     return 0;
51 }
```

Тесты:

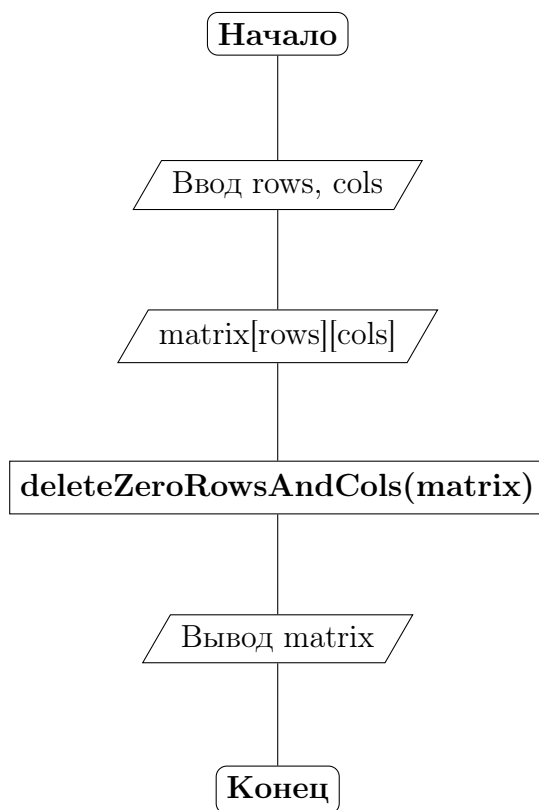
```
1
2 void test8_17_1() {
3     struct circle v[3] = {
4         {5, 6, 5},
5         {5, 20, 8},
6         {25, 15, 7}
7     };
8
9     int* res1 = findMinPairCircle(v, 3);
10
11     assert(res1[0] == 1);
12     assert(res1[1] == 2);
13
14     free(res1);
15 }
16
17 void test8_17_2() {
18     struct circle v[6] = {
19         {100, 100, 20},
20         {150, 160, 32},
21         {140, 110, 15},
22         {110, 190, 10},
23         {120, 180, 5},
24         {115, 125, 10}
25     };
26
27     int* res1 = findMinPairCircle(v, 6);
28
29     assert(res1[0] == 2);
30     assert(res1[1] == 5);
31
32     free(res1);
33 }
34
35
36 void test8_17_3() {
37     struct circle v[4] = {
38         {1, 0, 4},
39         {0, 0, 1},
40         {0, 2, 2},
41         {0, 0, 3}
42     };
43
44     int* res1 = findMinPairCircle(v, 4);
45
46     assert(res1[0] == 1);
47     assert(res1[1] == 4);
48
49     free(res1);
50 }
51
52 void test() {
53     test8_17_1();
54     test8_17_2();
55     test8_17_3();
56 }
```

```
1  
2 int main() {  
3     test();  
4  
5     return 42;  
6 }
```

1.9 Тема 9. Линейная алгебра и сжатие информации

В матрице удалить нулевые строки и столбцы. Результат вывести в виде матрицы.

Блок - схема:



Обоснование: Данная программа реализует несколько функций для работы с матрицами: `getMatrix`, `createMatrixFromArray`, `initMatrix` и `deleteMatrix`, а также определяет структуру `Matrix` для представления матриц.

Код программы:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <assert.h>
4
5 typedef struct{
6     int** data;
7     int rows;
8     int cols;
9 } Matrix;
10
11 //Создает и инициализирует новую матрицу с заданным числом строк и столбцов.
12 Matrix getMatrix(int nRows, int nCols) {
13     int **values = (int **) malloc(sizeof(int *) * nRows);
14     for (int i = 0; i < nRows; i++) {
15         values[i] = (int *) malloc(sizeof(int) * nCols);
16     }
17
18     return (Matrix) {values, nRows, nCols};
19 }
20
21 //Создает и инициализирует новую матрицу на основе одномерного массива элементов.
22 Matrix createMatrixFromArray(const int *a, int nRows, int nCols) {
23     Matrix m = getMatrix(nRows, nCols);
24
25     int k = 0;
26     for (int i = 0; i < nRows; i++)
27         for (int j = 0; j < nCols; j++)
28             m.data[i][j] = a[k++];
29
30     return m;
31 }
32 //Инициализирует новую матрицу с заданным числом строк и столбцов.
33 Matrix initMatrix(int rows, int cols) {
34     Matrix matrix;
35     matrix.data = (int**)malloc(rows * sizeof(int*));
36     for (int i = 0; i < rows; i++) {
37         matrix.data[i] = (int*)malloc(cols * sizeof(int));
38     }
39     matrix.rows = rows;
40     matrix.cols = cols;
41     return matrix;
42 }
43 //Освобождает память, занимаемую матрицей, и устанавливает ее размеры в 0.
44 void deleteMatrix(Matrix* matrix) {
45     for (int i = 0; i < matrix->rows; i++) {
46         free(matrix->data[i]);
47     }
48     free(matrix->data);
49     matrix->rows = 0;
50     matrix->cols = 0;
51 }
```

```

1 //Подсчитывает количество нулей в указанной строке матрицы.
2 int findZeroCount(Matrix* matrix, int row) {
3     int count = 0;
4     for (int j = 0; j < matrix->cols; j++) {
5         if (matrix->data[row][j] == 0) {
6             count++;
7         }
8     }
9     return count;
10 }
11
12 //Удаляет столбец с указанным индексом из матрицы.
13 void deleteColumn(Matrix* matrix, int column) {
14     for (int i = 0; i < matrix->rows; i++) {
15         for (int j = column; j < matrix->cols - 1; j++) {
16             matrix->data[i][j] = matrix->data[i][j + 1];
17         }
18     }
19     matrix->cols--;
20 }
21
22 //Находит индекс первого столбца, содержащего только нули.
23 int findZerosColumn(Matrix* matrix) {
24     for (int i = 0; i < matrix->cols; i++) {
25         int count = 0;
26         for (int j = 0; j < matrix->rows; j++) {
27             if (matrix->data[j][i] == 0) {
28                 count++;
29             }
30         }
31         if (count == matrix->rows) {
32             return i;
33         }
34     }
35     return matrix->cols;
36 }
37
38 // Удаляет все строки и столбцы матрицы, содержащие только нули.
39 void deleteRowColZero(Matrix* matrix) {
40     int pos = findZerosColumn(matrix);
41     while (pos != matrix->cols) {
42         deleteColumn(matrix, findZerosColumn(matrix));
43         pos = findZerosColumn(matrix);
44     }
45
46     for (int i = 0; i < matrix->rows; i++) {
47         if (findZeroCount(matrix, i) == matrix->cols) {
48             for (int j = i; j < matrix->rows - 1; j++) {
49                 for (int k = 0; k < matrix->cols; k++) {
50                     matrix->data[j][k] = matrix->data[j + 1][k];
51                 }
52             }
53             matrix->rows--;
54             i--;
55         }
56     }
57 }

```

Тесты:

```
1 void test9_17_1() {
2     Matrix matrix = createMatrixFromArray((int[]) {{3, 7, 2, 0},
3                                                    {4, 3, 6, 0},
4                                                    {0, 0, 0, 0},
5                                                    {5, 8, 90, 0}}, 4, 4);
6
7     Matrix res = createMatrixFromArray((int[]) {{3, 7, 2},
8                                                  {4, 3, 6},
9                                                  {5, 8, 90}}, 3, 3);
10
11
12     deleteRowColZero(&matrix);
13
14     assert(matrix.rows == res.rows);
15     assert(matrix.cols == res.cols);
16
17     for (int i = 0; i < matrix.rows; i++) {
18         for (int j = 0; j < matrix.cols; j++) {
19             assert(matrix.data[i][j] == res.data[i][j]);
20         }
21     }
22
23     deleteMatrix(&matrix);
24     deleteMatrix(&res);
25 }
26
27 void test9_17_2() {
28     Matrix matrix = createMatrixFromArray((int[]) {{0, 0, 0, 0},
29                                                    {0, 0, 0, 0},
30                                                    {0, 0, 0, 0},
31                                                    {0, 0, 0, 0}}, 4, 4);
32
33     deleteRowColZero(&matrix);
34
35     assert(matrix.rows == 0);
36     assert(matrix.cols == 0);
37
38     deleteMatrix(&matrix);
39 }
```

```

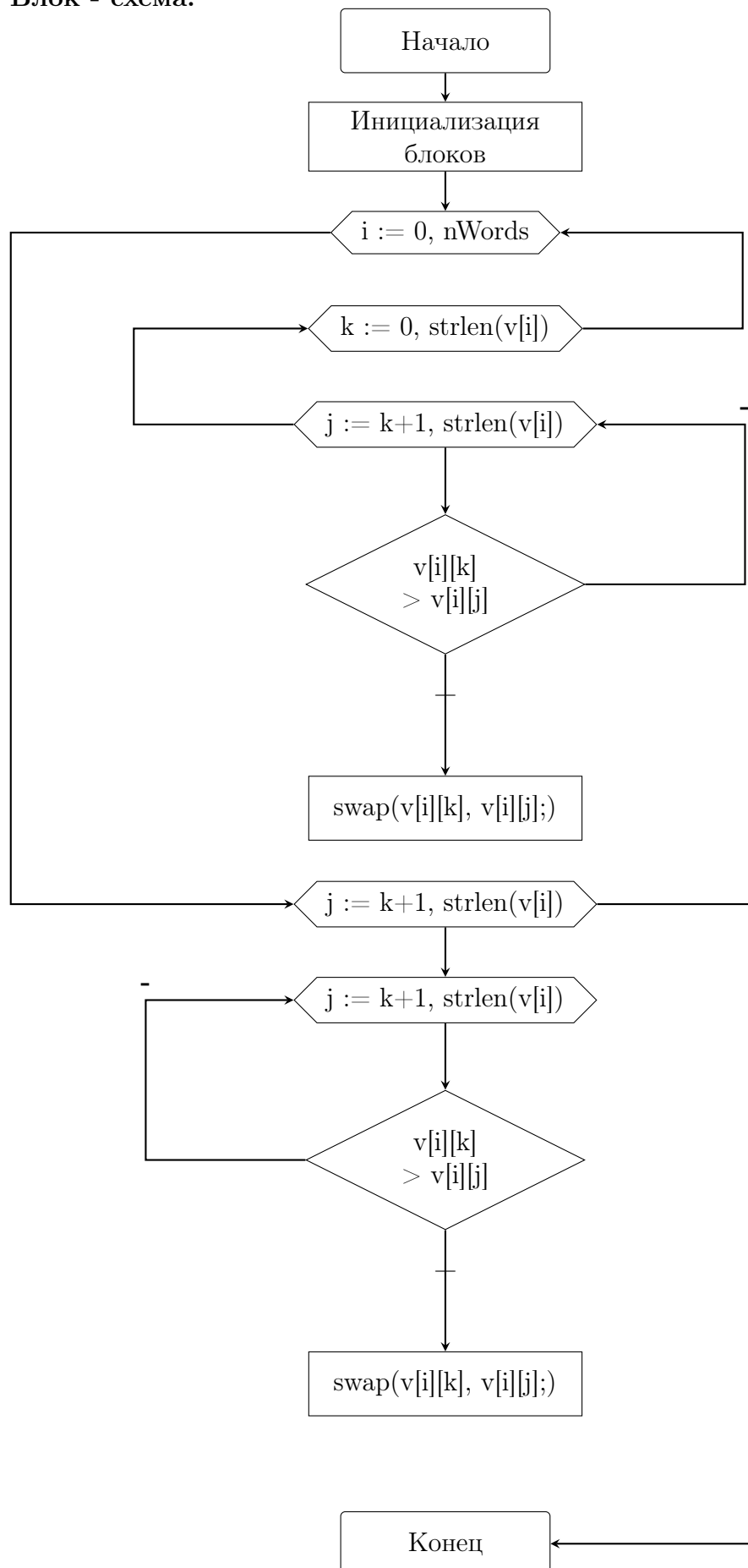
1 void test9_17_3() {
2     Matrix matrix = createMatrixFromArray((int []) {{0, 0, 0, 0},
3                                                     {4, 0, 6, 0},
4                                                     {0, 0, 0, 0},
5                                                     {5, 0, 90, 0}}, 4, 4);
6
7     Matrix res = createMatrixFromArray((int []) {{4, 6},
8                                                     {5, 90}}, 2, 2);
9
10    deleteRowColZero(&matrix);
11
12    assert(matrix.rows == res.rows);
13    assert(matrix.cols == res.cols);
14
15    for (int i = 0; i < matrix.rows; i++) {
16        for (int j = 0; j < matrix.cols; j++) {
17            assert(matrix.data[i][j] == res.data[i][j]);
18        }
19    }
20
21    deleteMatrix(&matrix);
22    deleteMatrix(&res);
23 }
24
25 void test() {
26     test9_17_1();
27     test9_17_2();
28     test9_17_3();
29 }
30
31 int main() {
32     test();
33
34     return 42;
35 }

```

1.10 Тема 10. Алгоритм обработки символьной информации

В имеющемся словаре найти группы слов, записанных одними и теми же буквами и отличающиеся только их порядком, то есть перестановкой, например, (КОМАР, КОРМА).

Блок - схема:



Обоснование: Данная программа реализует функции `inputWords`, `printWordPass` и `main`, которые взаимодействуют с пользователем, считывают слова, сортируют их и выводят пары слов, которые являются анаграммами.

Код программы:

```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 void inputWords(char v[][100], int nWords) {
6     int l = 0;
7     while (1) {
8         char c;
9         scanf("%c", &c);
10
11         if (isalpha(c))
12             v[l][strlen(v[l])] = c;
13         else if (c == ',')
14             l++;
15         else if (c == '\n')
16             break;
17     }
18 }
19
20 void printWordPass(char v[][100], char out[][100], int i, int wordLength) {
21     for (int k = 0; k < wordLength; ++k) {
22         printf("%c ", out[i][k]);
23     }
24 }
25
26 int main() {
27     int nWords;
28     scanf("%d", &nWords);
29
30     getchar();
31     char v[nWords][100];
32
33     inputWords(v, nWords);
34
35     char out[nWords][100];
36     memcpy(out, v, sizeof(v));
37
38     for (int i = 0; i < nWords; ++i) {
39         for (int k = 0; k < strlen(v[i]); ++k) {
40             for (int j = k + 1; j < strlen(v[i]); ++j) {
41                 if (v[i][k] > v[i][j]) {
42                     char temp = v[i][k];
43                     v[i][k] = v[i][j];
44                     v[i][j] = temp;
45                 }
46             }
47         }
48     }
```

```

1
2     for (int i = 0; i < nWords; ++i) {
3         for (int j = i + 1; j < nWords; ++j) {
4             if (strcmp(v[i], v[j]) == 0) {
5                 printWordPass(v, out, i, strlen(v[0]));
6                 printf("| ");
7                 printWordPass(v, out, j, strlen(v[0]));
8                 printf("\n");
9             }
10        }
11    }
12
13    return 42;
14}

```

Результаты работы программы:

```

4
amma tama opapa opapa
a m m a | m a m a
o p a n | o n a p

Process finished with exit code 42

```

1.11 Тема 11. Аналитическая геометрия

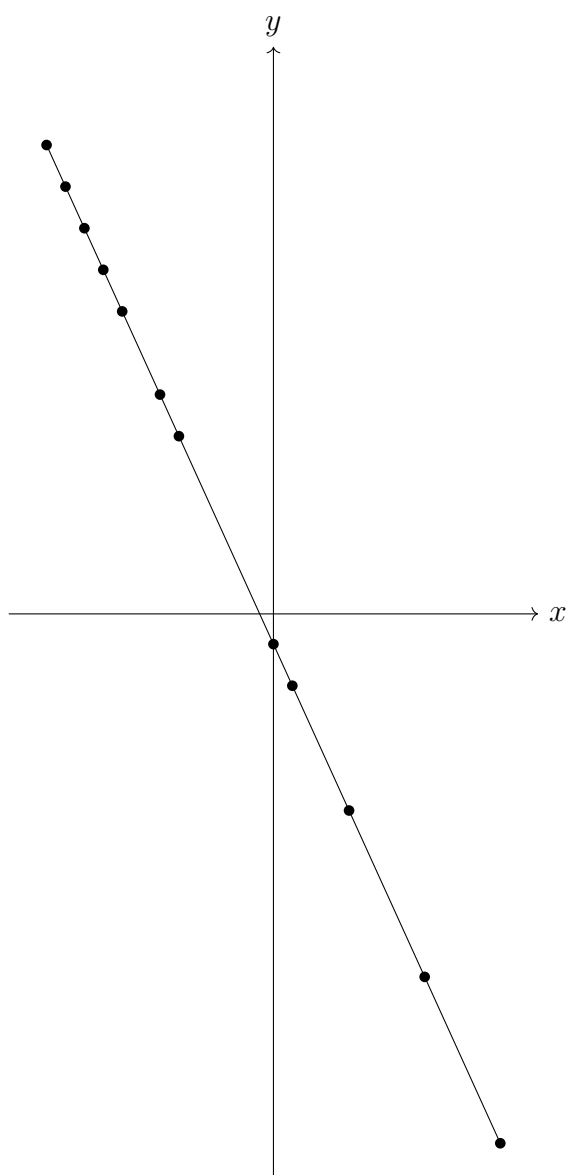
Построить прямую, проходящую через две данные точки:
 $M(3; -7)$ и $(-2; 4)$ в диапазоне $x \in [-3; 3]$ с шагом $\Delta = 0,25$.

$$\begin{aligned}\frac{y-4}{-7-4} &= \frac{x+2}{3+2} \\ \frac{y-4}{-11} &= \frac{x+2}{5} \\ y &= \frac{-11x-2}{5}\end{aligned}$$

Составляем таблицу:

x	y
-3	6.2
-2.75	5.65
-2.5	5.1
-2.25	4.55
-2	4
-1.5	2.9
-1.25	2.35
0	-0.4
0.25	-0.95
1	-2.6
2	-4.8
3	-7

График:



1.12 Тема 12. Кривые второго порядка на плоскости

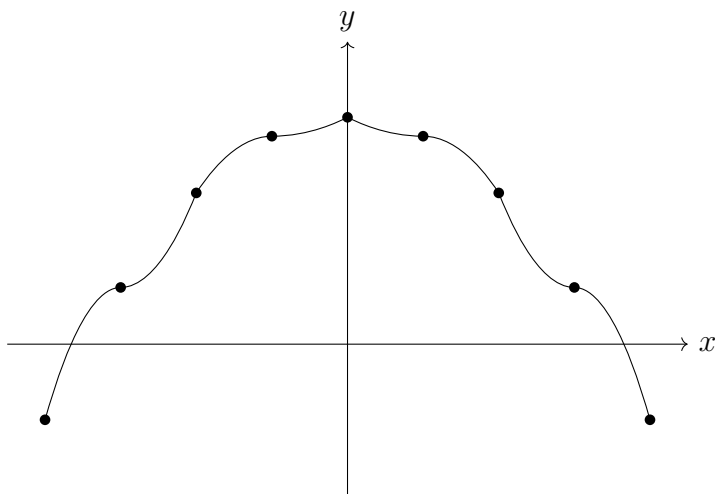
Постройте множество точек, одинаково удалённых от $F(0; 2)$ и $y = 4$ от прямой. Найдите точки пересечения этой кривой с осями координат и постройте её. Диапазон и шаг выберите самостоятельно.

$$\begin{aligned}x^2 + (y - 2)^2 &= (y - 4)^2 \\x^2 + (y - 2 - y + 4)(y - 2 + y - 4) &= 0 \\x^2 + 2(2y - 6) &= 0 \\y &= 3 - \frac{x^2}{4}\end{aligned}$$

Составляем таблицу:

x	y
-4	-1
-3	0.75
-2	2
-1	2.75
0	3
1	2.75
2	2
3	0.75
4	-1

График



Точки пересечения: $(-\sqrt{12}; 0) \cup (\sqrt{12}; 0) \cup (0; 3)$

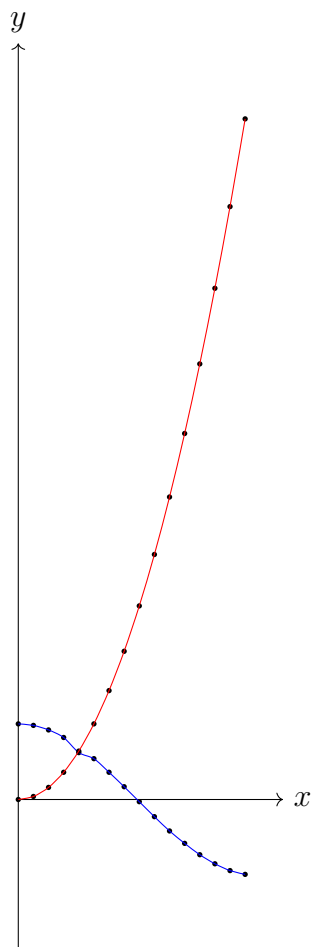
1.13 Тема 13. Графическое решение систем уравнений

$$\begin{cases} y = \cos x \\ 0 \leq x \leq 3, \Delta = 0, 2. \\ y = x^2 \end{cases}$$

Таблица:

x	y_1	y_2
0.2	0.98	0.04
0.4	0.92	0.16
0.6	0.82	0.36
0.8	0.9	0.64
1	0.54	1
1.2	0.36	1.44
1.4	0.16	1.96
1.6	-0.02	2.56
1.8	-0.22	3.24
2	-0.41	4
2.2	-0.58	4.84
2.4	-0.73	5.76
2.6	-0.85	6.76
2.8	-0.94	7.84
3	-0.98	9

График



Точка пересечения: (0.824; 0.679)

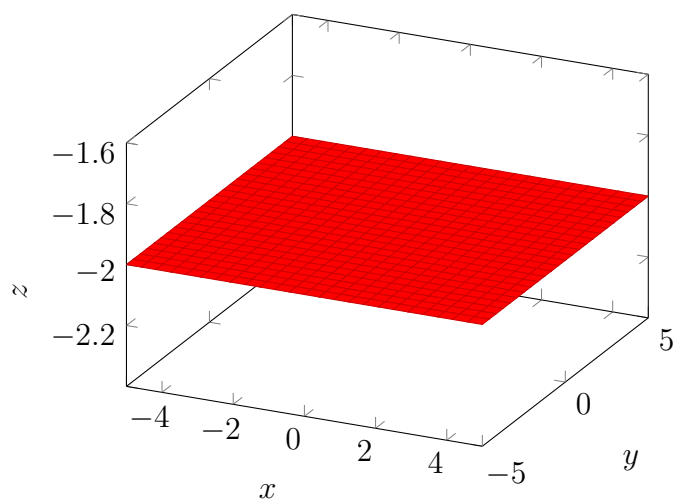
1.14 Тема 14. Плоскость в трехмерном пространстве

Построить плоскость, параллельную плоскости Oxy и пересекающую ось Oz в точке $M(0; 0; 2)$, при $-6 \leq x \leq 1$ с шагом $\Delta = 1$, $-3 \leq y \leq 8$ с шагом $\Delta = 1$.

Таблица:

x/y	0	1	2	3	4	5	6
-6	-2	-2	-2	-2	-2	-2	-2
-5	-2	-2	-2	-2	-2	-2	-2
-4	-2	-2	-2	-2	-2	-2	-2
-3	-2	-2	-2	-2	-2	-2	-2
-2	-2	-2	-2	-2	-2	-2	-2
-1	-2	-2	-2	-2	-2	-2	-2
0	-2	-2	-2	-2	-2	-2	-2

График:



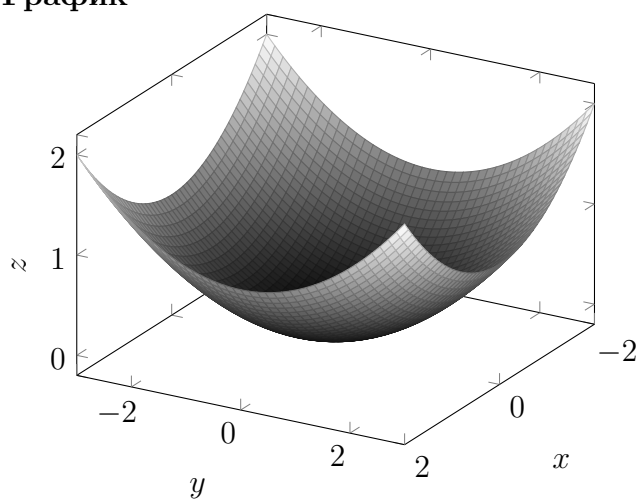
1.15 Тема 15. Поверхность второго порядка в трехмерном пространстве

Построить верхнюю часть конуса, заданного уравнением $\frac{x^2}{4} + \frac{y^2}{9} - \frac{z^2}{4} = 0$, лежащую в диапазоне $-2 \leq x \leq 2$ с шагом $\Delta = 0,5$, и $-3 \leq y \leq 3$ с шагом $\Delta = 1$.

Составляем таблицу:

x/y	-3	-2	-1	0	1	2	3
-2	0.7	0.6	0.52	0.5	0.52	0.6	0.7
-1.5	0.6	0.5	0.4	0.37	0.6	0.5	0.4
-1	0.55	0.41	0.3	0.25	0.55	0.41	0.3
-0.5	0.5	0.3	0.2	0.12	0.5	0.3	0.2
0	0.5	0.33	0.16	0	0.5	0.33	0.16
0.5	0.5	0.3	0.2	0.12	0.5	0.3	0.2
1	0.55	0.41	0.3	0.25	0.55	0.41	0.3
1.5	0.6	0.5	0.4	0.37	0.6	0.5	0.4
2	0.7	0.6	0.52	0.5	0.52	0.6	0.7

График



1.16 Заключение

В ходе данной практики по программированию были рассмотрены различные программы и функции, каждая из которых выполняет определенную задачу. Программы включают в себя работу с массивами, структурами, матрицами, строками, а также использование различных библиотек для решения задач.

В процессе работы с каждой программой было проведено обоснование ее работы, а также объяснено, как и для чего использовать соответствующие функции и структуры данных. Обоснование включало в себя описание целей программы, объяснение алгоритмов и детальное описание входных и выходных данных.

Практика по программированию позволила углубить понимание различных концепций и инструментов программирования, а также развить навыки решения задач с использованием языка программирования C. В процессе работы было уделено внимание не только самим программам, но и важности четкого кодирования, структурирования и комментирования кода для повышения его читаемости и поддерживаемости.

В заключение, практика по программированию позволила углубить знания и навыки программирования на языке C, а также приобрести опыт в решении различных задач с использованием различных алгоритмов и структур данных. Эти навыки могут быть применены в различных областях, где требуется разработка программного обеспечения, а также в дальнейшем изучении и практике программирования.